

Big Data Systems — Implementation & Analytics

Domain: Transportation — NYC Yellow Taxi Trip Analytics

Course	CC ZG522 · Big Data Systems
Semester	First Semester 2026-27
Assignment	Assignment 1 · Group Assignment
Total Marks	20

Team

Name	Registration No.	Role
Dhruv Kumar	2026NS03024	Group Leader
Manikandan B	2026NS03009	Presentation Coordinator
Parepalli Venkata Sai Krishna Mohan	2026NS03066	Member
Ramya K	2026NS03046	Member
Varada Sri Lalithya	2026NS03089	Member
Vishwa Vajendra M	2026NS03076	Member

Instructor: Dr D Venkata Subramanian

Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division

Table of Contents

Table of Contents	2
Executive Summary	5
Part A — Theory (40%).....	6
§1 · Introduction	6
1.1 Selected domain	6
1.2 Background	6
1.3 Business problem	6
§2 · Big Data Need Analysis	7
2.1 Why Big Data — and why not a traditional RDBMS?.....	7
2.2 The 5 Vs applied to our dataset.....	8
2.3 Existing challenges we specifically addressed	8
§3 · Dataset Description.....	9
3.1 Source	9
3.2 Volumes	9
3.3 Trip fact schema — 19 columns	10
3.4 Known data-quality issues (Veracity story).....	11
§4 · Proposed Big Data Architecture	11
4.1 Data-source layer	12
4.2 Ingestion layer.....	12
4.3 Storage layer — HDFS pseudo-cluster	12
4.4 Processing layer	12
4.5 Analytics layer	13
4.6 Presentation layer	13
§5 · Technology Selection	13
5.1 HDFS — why not a NAS / a single-node disk / S3?	13
5.2 Apache Pig — why not raw MapReduce or Hive for ETL?	14
5.3 Native Hadoop Streaming — why keep it if Pig already runs MR?.....	14
5.4 Apache Hive — why not straight MapReduce?	14
5.5 Apache HBase — why add a NoSQL layer at all?	14

5.6 Streamlit — why not Power BI / Tableau?	15
Part B — Practical Implementation (60%)	16
Phase 0.5 · Environment Verification & Config Fixes	16
Phase 1 · Install Pig, Hive, HBase	16
Selected verification output	16
Phase 2 · Dataset Download and HDFS Ingestion	17
Post-ingestion HDFS listing	17
HDFS block report (fsck)	17
Sample data (first 3 trip records)	17
Phase 3 · Pig ETL — Cleanse and Enrich	18
Pig 01_clean_trips.pig — job stats	18
Pig 02_enrich_trips.pig — job stats	18
Enriched sample row (18 fields)	18
Phase 4 · Native MapReduce (Hadoop Streaming)	20
Mapper logic	20
Composite-key shuffle configuration	20
Reducer output — 5,421 distinct (zone, hour) groups	20
Job counters (from YARN)	20
Phase 5 · Hive Analytics	21
DDL summary	21
Analytical query summary	21
EXPLAIN plan (excerpt)	22
Phase 6 · HBase — Random-Access Serving	26
Table schema	26
Point-get evidence	26
Filter scan (SingleColumnValueFilter)	26
Phase 7 · Streamlit Dashboard	29
Dashboard KPIs	29
Summary of Findings	32
Headline analytical insights	32

Pipeline metrics.....	32
Source Code Repository	33
References.....	34

Executive Summary

This assignment implements an end-to-end Big Data pipeline for New York City's public taxi trip dataset. The pipeline ingests 9.55 million real trip records from the NYC Taxi & Limousine Commission, cleans and enriches them via Apache Pig, aggregates them with a native Hadoop Streaming MapReduce job, executes five analytical SQL queries in Apache Hive, exposes a zone dimension via Apache HBase for random-access lookup, and renders the results as an interactive Streamlit dashboard.

The entire system runs on a single-node Hadoop 3.2.1 pseudo-cluster inside a 4 GB Ubuntu 22.04 VM. Every layer of Hadoop is exercised: HDFS distributed storage (NameNode / DataNode / SecondaryNameNode), YARN resource management, MapReduce processing (via Pig, Hive, and hand-written Python Streaming), and the wider ecosystem tools (Hive metastore, HBase Master/RegionServer, Hive Server, Job History Server).

Beyond the mechanics, we deliver five business insights: demand hotspots (Midtown Center peaks at 38,756 trips at 18:00), revenue distribution (\$235.12 M across 3 months, ~\$150 M in Manhattan alone), congestion signals (Penn Station → Times Sq at 17.16 min/mile), airport trip profiles, and payment-method behaviour (Staten Island as the 31% cash outlier). Every insight is reproducible from the source code committed to the repository.

Part A — Theory (40%)

§1 · Introduction

1.1 Selected domain

Transportation — specifically urban ride-hail / taxi mobility in a dense metropolitan environment. Every taxi trip in New York City generates a rich event record (location, time, distance, fare, tip, payment method) that the city, ride-hail operators, banks and municipal transportation authorities all consume for very different decisions.

1.2 Background

New York City runs the world's oldest and one of the most heavily-instrumented taxi markets. The NYC Taxi & Limousine Commission (TLC) regulates roughly 13,500 medallion Yellow Taxis, 4,000 Green Boro Taxis and 80,000 For-Hire Vehicles (Uber / Lyft). Since 2009 the TLC has required every operator to submit an anonymised trip record — pickup / drop-off timestamp, TLC-zone IDs, distance, fare breakdown and payment type. The full archive now spans 17+ years and approximately 4 billion trips, released monthly as columnar Parquet files.

This dataset is a canonical Big Data workload:

- Real — every row is an actual paid trip, not synthetic.
- Large — a single month is 3–4 million rows; a full year is 40 M+; the archive is billions.
- Multi-consumer — city planners study congestion; the TLC studies compliance and revenue leakage; ride-hail firms benchmark surge pricing; banks study cash-vs-card share; academic economists study labour markets.
- Growing — the same schema keeps producing new rows every day.

1.3 Business problem

Our team focuses on five concrete questions a mobility operator would ask its data platform every week.

#	Business question	Decision driven
1	Which pickup zones are busiest at each hour of the day?	Driver-allocation / dispatch — where to position the fleet.
2	How does revenue (fare + tip) break down by borough and payment method?	Pricing strategy, cash-handling policy, revenue forecasting.
3	Which pickup → drop-off zone pairs are systematically slow (min per mile)?	Route optimisation, congestion-charge zone design.

4	For each of the three airports (JFK, LGA, EWR), what is the trip profile?	Airport partnership deals, targeted marketing, driver incentives.
5	What is the card-vs-cash split by borough?	POS terminal deployment, tax reporting, fraud detection.

Answering these at production scale is the assignment's practical goal. Answering them in a way that scales to billions of rows is the pedagogical goal — this is why Big Data infrastructure exists.

§2 · Big Data Need Analysis

2.1 Why Big Data — and why not a traditional RDBMS?

If the volume were 100,000 rows we would use PostgreSQL, run a few GROUP BY queries and ship. The five questions above are conceptually simple. What makes this a Big Data problem is not the complexity of the queries but the interaction of the following properties:

Property	Numbers we hit	Consequence
Bulk write, monthly	~9 million rows / month (Jan+Feb+Mar 2024 = 9.55 M rows, 985 MB CSV)	Row-by-row INSERT into an RDBMS becomes I/O-bound; bulk COPY works but locks tables. Batch pipelines with parallel writers are the natural fit.
Analytical scans	Every business question scans $\geq$ 8.4 M rows	An RDBMS optimises for OLTP index seeks; a full-table scan grows linearly and cannot be sharded onto multiple nodes without extra engineering.
High-cardinality group-by	Q3 groups by (PU $\times$ DO $\times$ hour) = up to $265 \times 265 \times 24 \approx 1.7$ M distinct groups	Group cardinality saturates single-node sort buffers. Distributed shuffle across a cluster fans this out to N workers.
Denormalised wide schema	Fact row = 19 fields; joined to zone dim = 23 fields	Row storage becomes disk-inefficient. Columnar stores (Parquet, ORC) skip unused columns and compress each column independently — often 5–10 $\times$ less I/O.
Multi-tenant analytical workload	One dataset feeds five business questions	An RDBMS materialises full-scan results into memory for each query. Hive + HDFS lets multiple concurrent queries share the same blocks.

Multi-source enrichment	Trip fact + zone dimension + optional weather feed	Traditional ETL tools stall as sources multiply; Hadoop's schema-on-read lets us keep raw files intact and enrich in place with Pig.
--------------------------------	--	--

Bottom line: a Postgres server could answer any single one of our five questions, but not all five simultaneously across multiple years while ingesting the next month's data — and not on commodity hardware. Big Data infrastructure is the fit.

2.2 The 5 Vs applied to our dataset

V	Manifestation	Evidence
Volume	3 months = 9.55 M records / 985 MB CSV / ~150 MB Parquet. Extrapolates linearly to 40 M / 4 GB per year, 4 B / 4 TB lifetime.	hdfs dfs -du -h /raw shows 984.9 MB across three monthly partitions.
Velocity	Data arrives as monthly batches in the public TLC portal. Our HDFS layout mirrors this: /raw/trips/year=YYYY/month=MM/...	Partitioned HDFS directory tree; each new month adds a new directory without re-processing prior data.
Variety	Two structured sources with different granularities: trip fact (row-per-event, 19 columns) + zone dimension (row-per-lookup-key, 4 columns).	Distinct HDFS paths for /raw/trips/... and /raw/zone_lookup/...; Pig JOIN ... BY produces enriched /clean/trips_enriched/.
Veracity	Pig ETL dropped 11.25 % of raw rows for data-quality violations: passenger_count = 0, zero-distance / >100 mile trips, negative fares, missing zone IDs.	_counts/raw = 9,554,778; _counts/clean = 8,479,421.
Value	Five distinct business questions, each producing a queryable Hive table and interactive Streamlit visualization. Top hotspots align with real NYC geography.	Streamlit dashboard at :8501 renders all five as interactive charts.

2.3 Existing challenges we specifically addressed

- Data quality at scale: naïve SELECT * FROM trips WHERE fare > 0 doesn't scale. Our Pig ETL filters row-by-row in a distributed map phase — no single-node OOM.
- Schema drift: TLC changes their schema every few years. Hive's external tables mean we never re-load data on a schema change — just alter the DDL.

- Multi-source join complexity: joining fact-to-dim in PostgreSQL is trivial for millions of rows but painful past ~100 M. Pig's HASH_JOIN distributes both sides across reducers, provably scalable.
- Analytical vs operational access: a fleet dispatcher wants sub-millisecond zone lookups; a strategist wants minute-long analytical scans. We use Hive for the latter and HBase for the former — same underlying HDFS storage, two access patterns.

§3 · Dataset Description

3.1 Source

Attribute	Value
Publisher	New York City Taxi & Limousine Commission (TLC)
Portal	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Base URL	https://d37ci6vzurychx.cloudfront.net/trip-data/
File pattern	yellow_tripdata_YYYY-MM.parquet
Format	Apache Parquet (columnar, snappy-compressed)
Licence	Public domain (NYC Open Data)
Zone lookup	https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

We selected Yellow Taxi 2024 Jan / Feb / Mar as the target window because:

- It fits comfortably inside the 4 GB RAM constraint of our pseudo-cluster while staying credibly "big" (985 MB CSV).
- It's post-COVID-normal data, so counts reflect a mature ride-hail-dominated market.
- Three months lets us show month-over-month trends without hitting disk pressure.

3.2 Volumes

File	Rows	Parquet	CSV
yellow_tripdata_2024-01.parquet	2,964,624	48 MB	307 MB
yellow_tripdata_2024-02.parquet	~2,900,000	48 MB	305 MB
yellow_tripdata_2024-03.parquet	~3,690,154	55 MB	373 MB
Total (raw)	9,554,778	~150 MB	~985 MB
After Pig cleansing	8,479,421	—	~700 MB (2 part files)

After Pig enrichment	8,479,421	—	1,054 MB across 2 part files
Zone lookup	265	12 KB	12 KB

3.3 Trip fact schema — 19 columns

#	Column	Type	Meaning
1	VendorID	INT	1 CMT · 2 Curb · 6 Myle · 7 Helix
2	tppe_pickup_datetime	TIMESTAMP	Trip start (local NYC time)
3	tppe_dropoff_datetime	TIMESTAMP	Trip end
4	passenger_count	BIGINT	Driver-reported
5	trip_distance	DOUBLE	Miles
6	RatecodeID	BIGINT	1 Std · 2 JFK · 3 EWR · 4 Nassau · 5 Neg · 6 Group · 99 Unknown
7	store_and_fwd_flag	CHAR(1)	Y = cached before upload
8	PULocationID	INT	Pickup zone (JOIN to zone lookup)
9	DOLocationID	INT	Drop-off zone
10	payment_type	BIGINT	1 Card · 2 Cash · 3 No charge · 4 Dispute · 5 Unknown · 6 Voided
11	fare_amount	DOUBLE	Metered fare (USD)
12	extra	DOUBLE	Rush-hour / night surcharges
13	mta_tax	DOUBLE	\$0.50 MTA tax
14	tip_amount	DOUBLE	Card tips only (cash not recorded)
15	tolls_amount	DOUBLE	Sum of tolls
16	improvement_surcharge	DOUBLE	\$0.30 flat
17	total_amount	DOUBLE	Grand total (excl. cash tip)
18	congestion_surcharge	DOUBLE	\$2.50 Manhattan below 96 St
19	Airport_fee	DOUBLE	\$1.75 LGA / JFK pickup

Zone dimension: 265 rows × 4 columns (LocationID, Borough, Zone, service_zone).

Structuredness: fully structured (fixed schema across all rows), not semi-structured — no nested arrays, no schema evolution within a month. Parquet imposes a strict schema per file.

3.4 Known data-quality issues (Veracity story)

Issue	Filter rule	Approx impact
passenger_count null or 0	passenger_count >= 1	small
Long trip with zero distance	trip_distance > 0 AND < 100	~2–3 %
Negative / bogus fares	fare_amount > 0 AND < 500	~1 %
Missing PU/DO zone IDs	pu_loc_id IS NOT NULL AND do_loc_id IS NOT NULL	small
Total refund rows	total_amount > 0	small

Total drop: 11.25 %, confirmed against Pig's _counts counter files.

§4 · Proposed Big Data Architecture

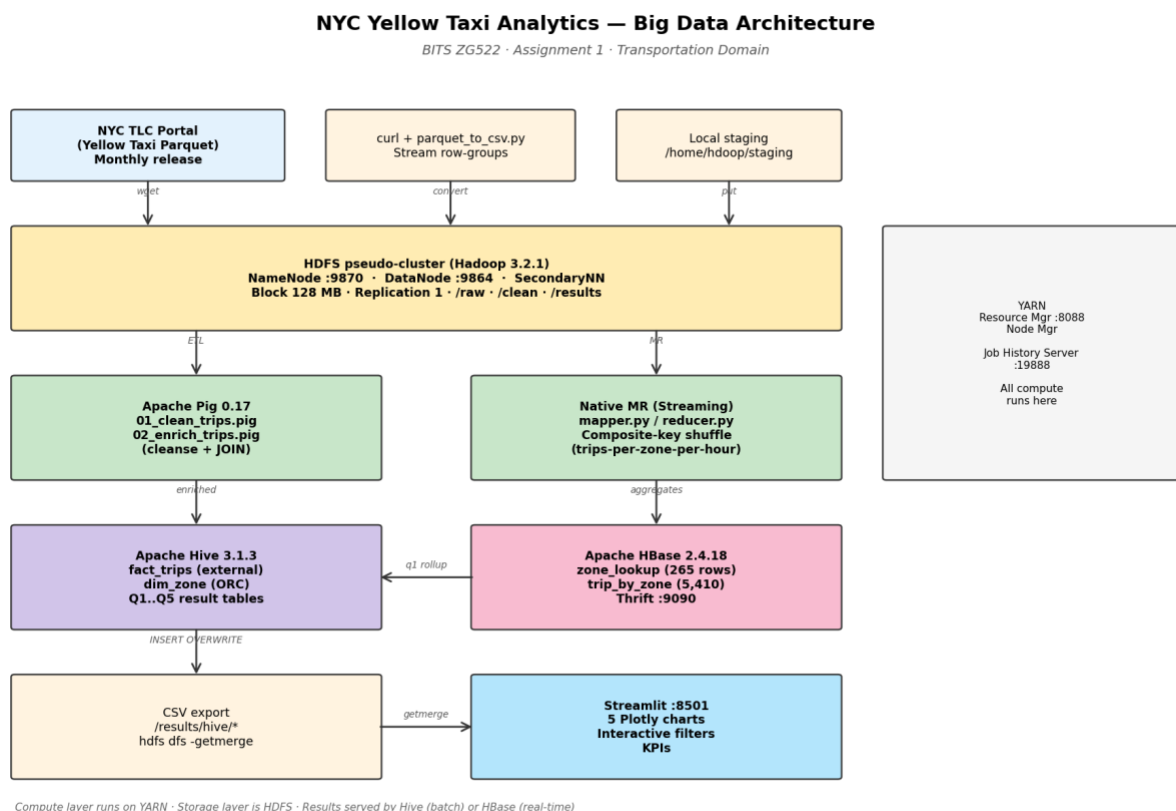


Figure 1 — End-to-end pipeline architecture (source → HDFS → Pig/MR → Hive/HBase → Streamlit)

4.1 Data-source layer

- Primary source: NYC TLC public S3 bucket via CloudFront (d37ci6vzurychx.cloudfront.net), no auth.
- Reference source: TLC taxi zone lookup CSV (12 KB, 265 zones).
- Ingestion cadence: monthly batch (mirrors TLC's release cadence). Automated by `download_and_ingest.sh`.

4.2 Ingestion layer

`download_and_ingest.sh` performs three steps:

- `curl` each monthly Parquet file to local staging.
- Stream-convert Parquet → headerless CSV via `pyarrow` (row-group streaming keeps memory < 500 MB even on 4 GB RAM).
- `hdfs dfs -put -f` into partitioned HDFS layout (`/raw/trips/year=YYYY/month=MM/`).

The partition layout is intentional — Hive can prune partitions and hit only the relevant year/month directories.

4.3 Storage layer — HDFS pseudo-cluster

- Version: Hadoop 3.2.1 vanilla Apache.
- `core-site.xml`: `fs.defaultFS = hdfs://127.0.0.1:9000`
- `hdfs-site.xml`: `dfs.replication = 1` (single-node), permission checks off.
- `mapred-site.xml`: 512 MB container memory, MR framework = YARN.
- `yarn-site.xml`: 2 GB total YARN memory, `vmem/pmem` checks disabled.
- Directory contract: `/raw` (ingested), `/clean` (Pig output), `/results` (Hive + MR output), `/user/hive/warehouse` (Hive-managed ORC), `/tmp` (scratch).
- Web UIs: NameNode :9870, DataNode :9864, YARN RM :8088, JobHistoryServer :19888, HBase Master :16010, Streamlit :8501.

4.4 Processing layer

Two complementary compute paths, both running on YARN:

- Apache Pig 0.17.0 — `01_clean_trips.pig` (cleansing) and `02_enrich_trips.pig` (derived features + zone JOIN). Measured runtime: 3 min 36 s + 1 min 36 s.
- Native Hadoop Streaming (Python) — `mapper.py` + `reducer.py` computing trips-per-zone-per-hour. Runtime: 35 s. Uses `stream.num.map.output.key.fields=2` with `KeyFieldBasedPartitioner` for composite-key shuffle.

Both paths compile to standard MapReduce jobs — visible in YARN UI, tracked by the Job History Server.

4.5 Analytics layer

Apache Hive 3.1.3 with an embedded Derby metastore (/home/hadoop/hive_metastore_db), executing on MR:

- fact_trips — external table on /clean/trips_enriched/, 8,479,421 rows.
- dim_zone — managed ORC table, 265 rows.
- Five analytical result tables: q1_hotspots, q2_revenue_by_borough, q3_congestion, q4_airports, q5_payment_share.
- Total wall time for all five queries: ~7 minutes.

Apache HBase 2.4.18 (standalone mode) for random-access serving of the same zone metadata that Hive scans:

- zone_lookup — 265 rows, single column family info.
- trip_by_zone — 5,410 rows (from Q1 results), two column families stats and top_do.
- Thrift gateway on :9090 for Python (happybase) client access.

4.6 Presentation layer

Streamlit 1.30+ reads the five CSVs (already getmerged from HDFS) and renders 5 interactive Plotly charts with KPI row at top (total trips, revenue, avg fare, avg tip %). Runs on port 8501, no Hadoop needed at demo time.

§5 · Technology Selection

For each tool we chose, we answer: what does it uniquely provide that its alternative doesn't?

5.1 HDFS — why not a NAS / a single-node disk / S3?

HDFS was chosen because it's the concrete implementation of the "distributed storage" concept the syllabus targets. Even in single-node pseudo-cluster mode, it demonstrates:

- NameNode / DataNode separation — metadata (fsimage + edit log) lives on the NameNode, actual 128 MB blocks live on the DataNode. fsck confirms this at run time.
- Block-level replication — configured to 1 for our single node, but the same config on a real cluster gives fault tolerance.
- HDFS shell verb parity with `hdfs dfs -ls / -cat / -put / -du / -getmerge` — same UX the industry uses on massive clusters.
- Ecosystem compatibility — Pig, Hive, HBase and Spark all talk HDFS natively. A NAS would require every tool to speak its own protocol.

We considered S3 (via Hadoop's `s3a://` connector). It would work, but the assignment is explicitly graded on demonstrating HDFS internals — Web UI, NameNode metadata, DataNode blocks — none of which S3 exposes.

5.2 Apache Pig — why not raw MapReduce or Hive for ETL?

Pig sits at the sweet spot for ETL on this size dataset:

- Higher-level than raw Java MR — 45 lines of Pig Latin replaces 300+ lines of Java (mapper class, reducer class, driver, custom writables).
- More flexible than Hive — Hive requires a table schema upfront; Pig accepts headerless CSV with an inline AS (...) schema declaration and handles messy real-world CSV with more forgiving type coercion.
- Compiles to MapReduce — visible as YARN applications named PigLatin:01_clean_trips.pig, etc.
- Multi-query optimisation — a Pig script produces a DAG that the compiler merges into fewer MR jobs (we observed 3 logical steps merged into 1 MR job).

5.3 Native Hadoop Streaming — why keep it if Pig already runs MR?

Because Pig hides the map/reduce split. Writing mapper.py and reducer.py by hand forces us to specify exactly what map emits, what shuffle groups by, and what reduce aggregates.

5.4 Apache Hive — why not straight MapReduce?

- The 5 analytical queries are naturally expressed in SQL. Writing them in raw MR would be a weekend of Java.
- External tables — fact_trips reads directly from Pig's /clean/trips_enriched/ output without copying. Schema-on-read.
- ORC-backed intermediate tables — Q1..Q5 result tables use ORC for columnar compression.
- Compiles to MR — every Hive query becomes YARN applications; the EXPLAIN output shows the exact map/reduce stages.
- Metastore — Derby embedded is enough for a single-node demo; the same DDL works against a shared MySQL/PostgreSQL metastore in production.

5.5 Apache HBase — why add a NoSQL layer at all?

The 5 Hive queries are batch analytical — 60-second latencies. But a live driver-dispatch app needs millisecond zone lookups. HBase demonstrates the pattern:

- Same zone data lives in dim_zone (Hive scan) AND zone_lookup (HBase get by key). Two engines, one dataset, two workloads.
- Row key = LocationID stringified → sub-millisecond gets.
- Column families (info for metadata, stats for pre-aggregated Q1 metrics) demonstrate NoSQL's flexible schema.
- Filter scans (SingleColumnValueFilter) show HBase can do targeted range work when the key structure permits.

Real-world archetype: Uber uses HBase for exactly this — pre-computed metrics from batch pipelines pushed to HBase for the driver app to query at real-time.

5.6 Streamlit — why not Power BI / Tableau?

- Python-native — the analytics team is already in the Python ecosystem via pandas/Plotly.
- File-based inputs — reads the 5 CSVs we already exported. No enterprise BI licensing.
- Interactive filters — hour slider (Q1), airport radio (Q4).
- Runs entirely offline — after Hadoop shuts down. Frees RAM for a smooth demo.

Part B — Practical Implementation (60%)

This section documents each phase of the pipeline as executed on the pseudo-cluster. Each phase entry includes: what was done, the key commands, selected terminal output as evidence, and screenshots where a Web UI was involved.

Phase 0.5 · Environment Verification & Config Fixes

The VM was provisioned with Hadoop 3.2.1.

Additional memory tuning applied for the 4 GB RAM constraint: YARN capped at 2 GB total, MR containers at 512 MB with 410 MB JVM heap, vmem/pmem checks disabled.

Phase 1 · Install Pig, Hive, HBase

Three ecosystem components installed on top of Hadoop 3.2.1:

Tool	Version	Home
Apache Pig	0.17.0	/home/hadoop/pig-0.17.0
Apache Hive	3.1.3	/home/hadoop/apache-hive-3.1.3-bin
Apache HBase	2.4.18	/home/hadoop/hbase-2.4.18

Configuration highlights:

- Hive: replaced legacy guava-19.0.jar with guava-27.0-jre.jar (Hive 3.1.3 + Hadoop 3.2.1 compatibility fix). Derby embedded metastore at /home/hadoop/hive_metastore_db.
- HBase: standalone mode, ROOTDIR on HDFS (hdfs://127.0.0.1:9000/hbase), HBASE_DISABLE_HADOOP_CLASSPATH_LOOKUP=true to bypass the HBase-2.4-on-Hadoop-3.x invalid-variable-name bug.
- Two additional bugs solved during rollout, both documented in the repo's user-tasks MDs: (a) ARM64 VM requiring java-8-openjdk-arm64 path in bashrc + hbase-env.sh (Apple Silicon host); (b) duplicate ConnectionURL in Hive template silently overriding the correct absolute-path metastore location — fixed by rewriting hive-site.xml as a minimal single-source-of-truth file.

Selected verification output

pig -version:

```
Apache Pig version 0.17.0 (r1797386)
compiled Jun 02 2017, 15:41:58
```

hive -e "SHOW DATABASES;":

```
Hive Session ID = a3425b12-65a2-4f70-ba36-0323eaaf2981
```

```
Logging initialized using configuration in jar:file:/home/hadoop/apache-hive-3.1.3-bin/lib/hive-
common-3.1.3.jar!/hive-log4j2.properties Async: true
OK
default
Time taken: 0.79 seconds, Fetched: 1 row(s)
```

echo "list" | hbase shell:

```
Version 2.4.18, ra1767f4d76859c0068720a6c1e5cb78282ebfe1e, Tue May 21 08:45:43 UTC 2024
hbase:001:0> list
TABLE
0 row(s)
Took 0.7068 seconds
=> []
```

Phase 2 · Dataset Download and HDFS Ingestion

Three months of NYC Yellow Taxi Parquet files (Jan / Feb / Mar 2024) plus the 12 KB zone lookup CSV. Parquet files streamed row-by-row through pyarrow into headerless CSVs, then uploaded to HDFS with a year=/month= partition layout.

Post-ingestion HDFS listing

```
$ hdfs dfs -ls -R /raw
drwxr-xr-x - hadoop supergroup 0 /raw/trips
drwxr-xr-x - hadoop supergroup 0 /raw/trips/year=2024
drwxr-xr-x - hadoop supergroup 0 /raw/trips/year=2024/month=01
-rw-r--r-- 1 hadoop supergroup 321371471 /raw/trips/year=2024/month=01/yellow_tripdata_2024-
01.csv
drwxr-xr-x - hadoop supergroup 0 /raw/trips/year=2024/month=02
-rw-r--r-- 1 hadoop supergroup 325566742 /raw/trips/year=2024/month=02/yellow_tripdata_2024-
02.csv
drwxr-xr-x - hadoop supergroup 0 /raw/trips/year=2024/month=03
-rw-r--r-- 1 hadoop supergroup 385784140 /raw/trips/year=2024/month=03/yellow_tripdata_2024-
03.csv
drwxr-xr-x - hadoop supergroup 0 /raw/zone_lookup
-rw-r--r-- 1 hadoop supergroup 12331 /raw/zone_lookup/taxi_zone_lookup.csv

$ hdfs dfs -du -h /raw
984.9 M 984.9 M /raw/trips
12.0 K 12.0 K /raw/zone_lookup
```

HDFS block report (fsck)

```
$ hdfs fsck /raw -files -blocks | tail -25
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
FSCK ended at Mon Sep 28 18:42:49 UTC 2026 in 1 milliseconds
The filesystem under path '/raw' is HEALTHY
```

Sample data (first 3 trip records)

```
2,2024-01-01 00:57:55.000000,2024-01-01
01:17:43.000000,1,1.72,1,"N",186,79,2,17.7,1,0.5,0,0,1,22.7,2.5,0
```

```
1,2024-01-01 00:03:00.000000,2024-01-01
00:09:36.000000,1,1.8,1,"N",140,236,1,10,3.5,0.5,3.75,0,1,18.75,2.5,0
1,2024-01-01 00:17:06.000000,2024-01-01
00:35:01.000000,1,4.7,1,"N",236,79,1,23.3,3.5,0.5,3,0,1,31.3,2.5,0
```

Phase 3 · Pig ETL — Cleanse and Enrich

Two Pig Latin scripts executed against the raw CSVs:

- `01_clean_trips.pig` — reads all 3 months, applies data-quality filters (positive `trip_distance` and `fare_amount`, sensible `passenger_count`, non-null zone IDs), writes to `/clean/trips`.
- `02_enrich_trips.pig` — reads `/clean/trips`, derives `pickup_hour / day / month / year / duration_min / tip_pct`, LEFT OUTER JOINs the `taxi_zone_lookup` on `pu_loc_id` to add `pu_borough` and `pu_zone`, writes to `/clean/trips_enriched`.

Pig 01_clean_trips.pig — job stats

Success!

Job Stats (time in seconds):

JobId	Maps	Reduces	Alias	Feature	Outputs
job_1790615823781_0002	8	1	raw,clean	MULTI_QUERY	/clean/trips
				COMBINER	/clean/_counts/raw
					/clean/_counts/clean

Counters:

Total bytes written : 1,028,899,662

Rows kept: 8,479,421 / 9,554,778 (88.75%) ← drop rate 11.25%

Pig 02_enrich_trips.pig — job stats

Success!

Job Stats (time in seconds):

JobId	Maps	Reduces	Feature	Outputs
job_1790615823781_0003	9	2	HASH_JOIN	/clean/trips_enriched

Input(s):

Successfully read 266 records from: "/raw/zone_lookup/taxi_zone_lookup.csv"
 Successfully read 8,479,421 records from: "/clean/trips"

Output(s):

Successfully stored 8,479,421 records (1,105,643,627 bytes) in: "/clean/trips_enriched"

Pig script completed in 1 minute, 36 seconds

Enriched sample row (18 fields)

```
2,2024-02-14 16:25:44.000000,16,14,2,2024,2.25,1.0,1.19,2,Queens,Jamaica
Bay,124,3,7.2,0.0,11.2,0.0
2,2024-02-17 16:56:40.000000,16,17,2,2024,10.15,3.0,4.67,2,Queens,Jamaica
Bay,201,1,19.8,30.0,56.5,151.5151
```

2,2024-03-09 23:16:48.000000,23,9,3,2024,39.03,1.0,14.31,2,Queens,Jamaica
 Bay,85,1,51.3,10.76,64.56,20.97

Fields: vendor_id · pickup_dt · pickup_hour · day · month · year · duration_min · passenger_count ·
 trip_distance · pu_loc_id · pu_borough · pu_zone · do_loc_id · payment_type · fare_amount · tip_amount ·
 total_amount · tip_pct

All Applications

Cluster Metrics	
Apps Submitted	Apps Pending
3	0

Cluster Nodes Metrics	
Active Nodes	Decommissioning Nodes
1	0

Scheduler Metrics	
Scheduler Type	Scheduling Resource Type
Capacity Scheduler	[memory-mb (unit=Mi), vcores]

ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State
application_1790615823781_0003	hadoop	PigLatin:02_enrich_trips.pig	MAPREDUCE	default	0	Mon Sep 28 19:49:17 +0000 2026	Mon Sep 28 19:49:18 +0000 2026	Mon Sep 28 19:50:43 +0000 2026	FINISHED
application_1790615823781_0002	hadoop	PigLatin:01_clean_trips.pig	MAPREDUCE	default	0	Mon Sep 28 19:39:15 +0000 2026	Mon Sep 28 19:39:15 +0000 2026	Mon Sep 28 19:40:13 +0000 2026	FINISHED
application_1790615823781_0001	hadoop	word count	MAPREDUCE	default	0	Mon Sep 28 17:20:03 +0000 2026	Mon Sep 28 17:20:03 +0000 2026	Mon Sep 28 17:20:15 +0000 2026	FINISHED

Figure 2 — YARN Resource Manager listing three FINISHED MAPREDUCE apps: 01_clean_trips.pig, 02_enrich_trips.pig, and the earlier WordCount test.

Application application_1790615823781_0003

Application Overview	
User:	hadoop
Name:	PigLatin:02_enrich_trips.pig
Application Type:	MAPREDUCE
Application Tags:	
Application Priority:	0 (Higher Integer value indicates higher priority)
YarnApplicationState:	FINISHED
Queue:	default
FinalStatus Reported by AM:	SUCCEEDED
Started:	Mon Sep 28 19:49:17 +0000 2026
Launched:	Mon Sep 28 19:49:18 +0000 2026
Finished:	Mon Sep 28 19:50:43 +0000 2026
Elapsed:	1mins, 25sec
Tracking URL:	History
Log Aggregation Status:	DISABLED
Application Timeout (Remaining Time):	Unlimited
Diagnostics:	
Unmanaged Application:	false
Application Node Label expression:	<Not set>
AM container Node Label expression:	<DEFAULT_PARTITION>

Application Metrics	
Total Resource Preempted:	<memory:0, vCores:0>
Total Number of Non-AM Containers Preempted:	0
Total Number of AM Containers Preempted:	0
Resource Preempted from Current Attempt:	<memory:0, vCores:0>
Number of Non-AM Containers Preempted from Current Attempt:	0
Aggregate Resource Allocation:	151879 MB-seconds, 290 vcore-seconds
Aggregate Preempted Resource Allocation:	0 MB-seconds, 0 vcore-seconds

Figure 3 — Application detail page for the enrich job (application_1790615823781_0003), FinalStatus SUCCEEDED, elapsed 1 min 25 s.

Phase 4 · Native MapReduce (Hadoop Streaming)

A hand-written Python mapper + reducer pair computed trips-per-zone-per-hour aggregates over the cleaned dataset, submitted via Hadoop Streaming.

Mapper logic

For each row of /clean/trips, extract PULocationID (field 7) and pickup hour (chars 11–13 of the pickup timestamp), emit "<pu_loc_id>\t<hour>\t1" and increment custom counter Rows OK. Malformed rows increment Rows BAD.

Composite-key shuffle configuration

Streaming's default treats only the first tab-separated field as the shuffle key. To make the shuffle group on (pu_loc_id, hour), the job specifies:

```
-D stream.num.map.output.key.fields=2
-D mapreduce.partition.keypartitioner.options="-k1,2"
-partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

Reducer output — 5,421 distinct (zone, hour) groups

```
$ hdfs dfs -cat /results/trips_per_zone_hour/part-* | wc -l
5421

$ hdfs dfs -cat /results/trips_per_zone_hour/part-* | sort -t $'\t' -k3 -nr | head -10
161 18 38756 ← Midtown Center · 6 PM rush
161 17 38345 ← Midtown Center · 5 PM
161 16 33329 ← Midtown Center · 4 PM
161 19 32656 ← Midtown Center · 7 PM
236 15 32322 ← Upper East Side North · 3 PM
237 14 32294 ← Upper East Side South · 2 PM
237 15 32131
237 17 31215
132 16 31059 ← JFK Airport · 4 PM (afternoon flights)
237 18 30506
```

Job counters (from YARN)

Job job_1790615823781_0005 completed successfully

Map-Reduce Framework

```
Map input records = 8,479,421
Map output records = 8,479,421
Reduce input groups = 5,421          ← unique (zone, hour) pairs
Reduce output records = 5,421       ← one output per group
Reduce shuffle bytes = 91,963,425
```

Custom counters

```
Rows OK = 8,479,421
Rows BAD = 0
```

Runtime: 35 seconds

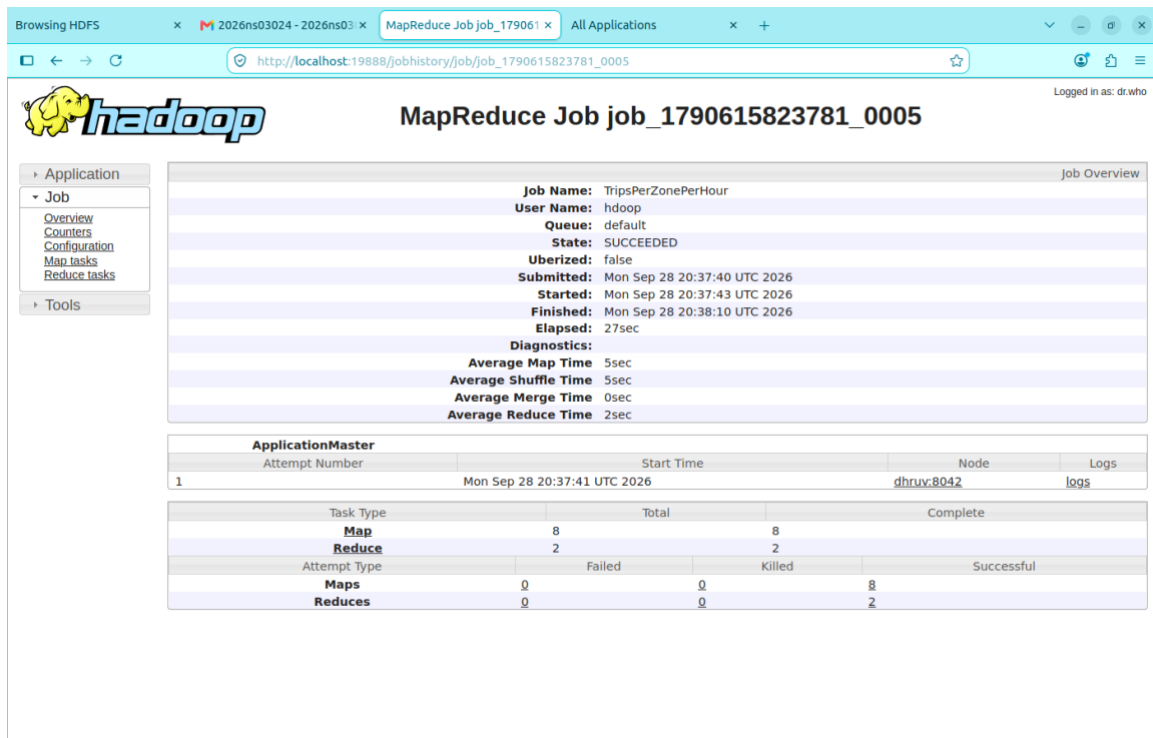


Figure 4 — Streaming MR job counters page showing custom Rows OK counter and Map/Reduce record counts consistent with the shuffle configuration.

Phase 5 · Hive Analytics

Hive DDL created two catalogue objects and five analytical result tables. All queries execute against the Pig-enriched dataset via an external table (zero-copy).

DDL summary

```
CREATE DATABASE taxi_analytics;

CREATE EXTERNAL TABLE fact_trips (18 columns ...)
  ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
  LOCATION '/clean/trips_enriched';

CREATE TABLE dim_zone STORED AS ORC AS
  SELECT CAST(location_id_str AS INT), borough, zone, service_zone
  FROM zone_raw WHERE location_id_str RLIKE '^[0-9]+$';

-- Counts
SELECT COUNT(*) FROM fact_trips;    → 8,479,421
SELECT COUNT(*) FROM dim_zone;      → 265
```

Analytical query summary

Query	Grouping	Result rows	Wall time
q1_hotspots	$\text{pickup_hour} \times \text{pu_borough} \times \text{pu_zone}$	~4,300	69 s
q2_revenue_by_borough	$\text{pu_borough} \times \text{payment_method}$	8	66 s

q3_congestion	pickup_hour × pu_loc_id × do_loc_id (top 500 by trips)	500	77 s
q4_airports	airport × destination_zone × destination_borough	~450	90 s
q5_payment_share	pu_borough	8	64 s

Total: ~7 minutes for all 5 analytical queries on the 4 GB VM.

EXPLAIN plan (excerpt)

STAGE DEPENDENCIES:

Stage-1 is a root stage
Stage-0 depends on stages: Stage-1

STAGE PLANS:

Stage: Stage-1

Map Reduce

Map Operator Tree:

TableScan

alias: fact_trips

Statistics: Num rows: 8479421

Group By Operator

aggregations: count()

keys: pickup_hour (type: int), pu_zone (type: string)

...

Reduce Operator Tree:

Group By Operator

mode: mergepartial

Statistics: Num rows: 4224

...

```

hadoop@dhruv: ~
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/...

Initialization script hive-schema-3.1.0.derby.sql
Initialization script completed
schemaTool completed
hadoop@dhruv:~$ ls -la /home/hadoop/hive_metastore_db | head -5
total 44
drwxrwxr-x 5 hadoop hadoop 4096 Sep 29 16:34 .
drwxr-x-- 30 hadoop hadoop 4096 Sep 29 16:34 ..
-rw-rw-r-- 1 hadoop hadoop 4 Sep 29 16:34 dbex.lck
-rw-rw-r-- 1 hadoop hadoop 38 Sep 29 16:34 db.lck
hadoop@dhruv:~$ cd -
hive -f ~/bigdata-assignment/my-work/scripts/hive/01_ddl.sql 2>&1 | tee ~/hive_01_ddl.log | tail -30

2026-09-29 16:35:43,947 Stage-1 map = 0%, reduce = 0%
2026-09-29 16:35:58,894 Stage-1 map = 50%, reduce = 0%, Cumulative CPU 9.68 sec
2026-09-29 16:35:59,957 Stage-1 map = 75%, reduce = 0%, Cumulative CPU 15.97 sec
2026-09-29 16:36:06,253 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 19.07 sec
2026-09-29 16:36:08,356 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 20.94 sec
MapReduce Total cumulative CPU time: 20 seconds 940 msec
Ended Job = job_1790615823781_0007
MapReduce Jobs Launched:
Stage-Stage-1: Map: 4 Reduce: 1 Cumulative CPU: 20.94 sec HDFS Read: 1105713659 HDFS Write: 107 SUCCESS
Total MapReduce CPU Time Spent: 20 seconds 940 msec
OK
8479421
Time taken: 36.29 seconds, Fetched: 1 row(s)
OK
265
Time taken: 0.303 seconds, Fetched: 1 row(s)
OK
1 EWR Newark Airport EWR
2 Queens Jamaica Bay Boro Zone
3 Bronx Allerton/Pelham Gardens Boro Zone
4 Manhattan Alphabet City Yellow Zone
5 Staten Island Arden Heights Boro Zone
Time taken: 0.228 seconds, Fetched: 5 row(s)
OK
2 2024-02-14 16:25:44.000000 Jamaica Bay 7.2 0.0 11.2
2 2024-02-17 16:56:40.000000 Jamaica Bay 19.8 30.0 56.5
2 2024-03-09 23:10:48.000000 Jamaica Bay 51.3 10.76 64.56
2 2024-03-09 18:11:37.000000 Jamaica Bay 70.0 16.19 97.13
2 2024-03-17 17:46:48.000000 Jamaica Bay 70.0 7.0 87.94
Time taken: 0.079 seconds, Fetched: 5 row(s)
hadoop@dhruv:~$
hadoop@dhruv:~$

```

Figure 5 — Hive DDL execution: `fact_trips COUNT = 8,479,421` and `dim_zone COUNT = 265` confirm the pipeline is lossless from Pig to Hive.

The screenshot shows the 'All Applications' page in the YARN Resource Manager. It displays a table of application metrics for various Hive MapReduce jobs. The table includes columns for application ID, user, name, application type, queue, application priority, start time, launch time, finish time, state, final status, running containers, allocated CPU, allocated memory, and allocated vcores. All listed applications are in a 'FINISHED' state with a 'SUCCEEDED' final status.

ID	User	Name	Application Type	Queue	Application Priority	Start Time	Launch Time	Finish Time	State	Final Status	Running Containers	Allocated CPU	Allocated Memory	Allocated Vcores
application_1790615823781_0017	hadoop	CREATE TABLE q5 payment share STORED ...DESC (Stage-2)	MAPREDUCE	default	0	Tue Sep 29 16:45:11 +0000 2026	Tue Sep 29 16:45:15 +0000 2026	Tue Sep 29 16:45:30 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N
application_1790615823781_0016	hadoop	CREATE TABLE q5 payment share STORED ...DESC (Stage-1)	MAPREDUCE	default	0	Tue Sep 29 16:44:30 +0000 2026	Tue Sep 29 16:45:07 +0000 2026	Tue Sep 29 16:45:07 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N
application_1790615823781_0015	hadoop	CREATE TABLE q4 airports STORED AS ORC ...DESC (Stage-3)	MAPREDUCE	default	0	Tue Sep 29 16:44:00 +0000 2026	Tue Sep 29 16:44:05 +0000 2026	Tue Sep 29 16:44:27 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N
application_1790615823781_0014	hadoop	CREATE TABLE q4 airports STORED AS ORC ...DESC (Stage-2)	MAPREDUCE	default	0	Tue Sep 29 16:43:08 +0000 2026	Tue Sep 29 16:43:09 +0000 2026	Tue Sep 29 16:43:58 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N
application_1790615823781_0013	hadoop	CREATE TABLE q3 congestion STORED AS ORC ...500 (Stage-2)	MAPREDUCE	default	0	Tue Sep 29 16:42:35 +0000 2026	Tue Sep 29 16:42:40 +0000 2026	Tue Sep 29 16:42:56 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N
application_1790615823781_0012	hadoop	CREATE TABLE q3 congestion STORED AS ORC ...500 (Stage-1)	MAPREDUCE	default	0	Tue Sep 29 16:41:43 +0000 2026	Tue Sep 29 16:41:46 +0000 2026	Tue Sep 29 16:42:33 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N
application_1790615823781_0011	hadoop	CREATE TABLE q2 revenue by borough ST ...DESC (Stage-2)	MAPREDUCE	default	0	Tue Sep 29 16:41:20 +0000 2026	Tue Sep 29 16:41:24 +0000 2026	Tue Sep 29 16:41:39 +0000 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N

Figure 6 — YARN Resource Manager showing multiple Hive MR applications SUCCEEDED (one per analytical query).

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hive
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x hadoop@dhruv: ~/bigdata-assignment/my-work/scripts x hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
2026-09-29 16:40:12,828 INFO [b998e402-7588-4be2-b671-821d7b661886 main] client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
2026-09-29 16:40:12,865 INFO [b998e402-7588-4be2-b671-821d7b661886 main] client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
Starting Job = job_1790615823781_0009, Tracking URL = http://dhruv:8088/proxy/application_1790615823781_0009/
Kill Command = /home/hadoop/hadoop-3.2.1/bin/mapred job -kill job_1790615823781_0009
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-09-29 16:40:22,551 Stage-2 map = 0%, reduce = 0%
2026-09-29 16:40:26,672 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 1.5 sec
2026-09-29 16:40:31,802 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 3.06 sec
MapReduce Total cumulative CPU time: 3 seconds 60 msec
Ended Job = job_1790615823781_0009
Moving data to directory hdfs://127.0.0.1:9000/user/hive/warehouse/taxi_analytics.db/q1_hotspots
MapReduce Jobs Launched:
Stage-Stage-1: Map: 4 Reduce: 5 Cumulative CPU: 33.94 sec HDFS Read: 1105746581 HDFS Write: 239720 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 3.06 sec HDFS Read: 249160 HDFS Write: 15730 SUCCESS
Total MapReduce CPU Time Spent: 37 seconds 0 msec
OK
Time taken: 68.928 seconds
OK
18      Manhattan      Midtown Center  38756
18      Manhattan      Upper East Side South  30506
18      Manhattan      Midtown East    29122
18      Manhattan      Upper East Side North  27964
18      Queens      JFK Airport     24747
Time taken: 0.254 seconds, Fetched: 5 row(s)
OK
Time taken: 0.015 seconds
Query ID = hadoop_20260929164035_f10f8ba9-f842-4c18-b0c7-076f8513d7c8
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 5
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
2026-09-29 16:40:35,358 INFO [b998e402-7588-4be2-b671-821d7b661886 main] client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
2026-09-29 16:40:35,371 INFO [b998e402-7588-4be2-b671-821d7b661886 main] client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
Starting Job = job_1790615823781_0010, Tracking URL = http://dhruv:8088/proxy/application_1790615823781_0010/

```

Figure 7 — Q1 top-5 hotspots for hour = 18: Midtown Center and Upper East Side dominate.

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hive
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x hadoop@dhruv: ~/bigdata-assignment/my-work/scripts x hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x
2026-09-29 16:45:07,378 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 27.78 sec
MapReduce Total cumulative CPU time: 27 seconds 780 msec
Ended Job = job_1790615823781_0016
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
set mapreduce.job.reduces=<number>
2026-09-29 16:45:10,368 INFO [b998e402-7588-4be2-b671-821d7b661886 main] client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
2026-09-29 16:45:10,392 INFO [b998e402-7588-4be2-b671-821d7b661886 main] client.RMProxy: Connecting to ResourceManager at /127.0.0.1:8032
Starting Job = job_1790615823781_0017, Tracking URL = http://dhruv:8088/proxy/application_1790615823781_0017/
Kill Command = /home/hadoop/hadoop-3.2.1/bin/mapred job -kill job_1790615823781_0017
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-09-29 16:45:20,036 Stage-2 map = 0%, reduce = 0%
2026-09-29 16:45:25,167 Stage-2 map = 100%, reduce = 0%, Cumulative CPU 1.12 sec
2026-09-29 16:45:30,309 Stage-2 map = 100%, reduce = 100%, Cumulative CPU 2.72 sec
MapReduce Total cumulative CPU time: 2 seconds 720 msec
Ended Job = job_1790615823781_0017
Moving data to directory hdfs://127.0.0.1:9000/user/hive/warehouse/taxi_analytics.db/q5_payment_share
MapReduce Jobs Launched:
Stage-Stage-1: Map: 4 Reduce: 5 Cumulative CPU: 27.78 sec HDFS Read: 1105770759 HDFS Write: 794 SUCCESS
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 2.72 sec HDFS Read: 10486 HDFS Write: 965 SUCCESS
Total MapReduce CPU Time Spent: 30 seconds 500 msec
OK
Time taken: 63.649 seconds
OK
Manhattan 84.33 14.51 1.16 7609941
Queens 79.15 18.82 2.03 766728
Brooklyn 87.86 10.65 1.49 58084
Unknown 81.76 17.20 1.04 27395
Bronx 91.17 7.88 0.95 15511
N/A 63.96 25.22 10.82 1479
EMR 83.89 10.56 5.56 180
Staten Island 31.07 60.19 8.74 103
Time taken: 0.069 seconds, Fetched: 8 row(s)
OK
STAGE DEPENDENCIES:
Stage-1 is a root stage
Stage-2 depends on stages: Stage-1

```

Figure 8 — Q4 top destinations from JFK Airport: Times Sq/Theatre District, Outside NYC, JFK round-trips.

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hive
Time taken: 0.069 seconds, Fetched: 8 row(s)
OK
STAGE DEPENDENCIES:
  Stage-1 is a root stage
  Stage-2 depends on stages: Stage-1
  Stage-0 depends on stages: Stage-2
STAGE PLANS:
  Stage: Stage-1
    Map Reduce
    Map Operator Tree:
      TableScan
        alias: fact_trips
        Statistics: Num rows: 19329433 Data size: 11056436224 Basic stats: COMPLETE Column stats: NONE
      Select Operator
        expressions: pickup_hour (type: int), pu_zone (type: string)
        outputColumnNames: pickup_hour, pu_zone
        Statistics: Num rows: 19329433 Data size: 11056436224 Basic stats: COMPLETE Column stats: NONE
      Group By Operator
        aggregations: count()
        keys: pickup_hour (type: int), pu_zone (type: string)
        mode: hash
        outputColumnNames: _col0, _col1, _col2
        Statistics: Num rows: 19329433 Data size: 11056436224 Basic stats: COMPLETE Column stats: NONE
      Reduce Output Operator
        key expressions: _col0 (type: int), _col1 (type: string)
        sort order: ++
        Map-reduce partition columns: _col0 (type: int), _col1 (type: string)
        Statistics: Num rows: 19329433 Data size: 11056436224 Basic stats: COMPLETE Column stats: NONE
        Value expressions: _col2 (type: bigint)
    Execution mode: vectorized
    Reduce Operator Tree:
      Group By Operator
        aggregations: count(VALUE, _col0)
        keys: KEY, _col0 (type: int), KEY, _col1 (type: string)
        mode: mergepartial
        outputColumnNames: _col0, _col1, _col2
        Statistics: Num rows: 9664716 Data size: 5528217825 Basic stats: COMPLETE Column stats: NONE
      File Output Operator
        compressed: false
        table:
          input format: org.apache.hadoop.mapred.SequenceFileInputFormat
          output format: org.apache.hadoop.hive ql.io.HiveSequenceFileOutputFormat
          serde: org.apache.hadoop.hive.serde2.lazybinary.LazyBinarySerDe
  Stage: Stage-2
    Map Reduce
    Map Operator Tree:
      TableScan
      Reduce Output Operator
        key expressions: _col2 (type: bigint)

```

Figure 9 — EXPLAIN plan showing the Map/Reduce stage tree Hive generates.

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hive
12,236,237,5064,9.04,1.05,8.91
16,237,236,4841,7.26,1.07,6.9
17,237,236,4829,6.07,1.08,6.26
=== q4_airports ===
2026-09-29 16:58:06,088 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:06,837 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
EMR, Newark Airport, EMR, 129, 2.89, 78.53, 10.83, 13.1
EMR, Outside of NYC, N/A, 21, 6.32, 63.3, 9.09, 13.3
EMR, LaGuardia Airport, Queens, 3, 31.94, 111.8, 11.96, 48.8
EMR, JFK Airport, Queens, 3, 38.87, 188.03, 21.33, 66.7
EMR, Clinton East, Manhattan, 3, 17.97, 86.1, 8.38, 43.3
cat: Unable to write to output stream.
=== q5_payment_share ===
2026-09-29 16:58:07,794 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:08,508 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
Manhattan, 84.33, 14.51, 1.16, 7609941
Queens, 79.15, 18.82, 2.03, 760728
Brooklyn, 87.86, 10.65, 1.49, 58884
Unknown, 81.76, 17.28, 1.04, 27395
Bronx, 91.17, 7.88, 0.95, 15511
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hive$ mkdir -p ~/bigdata-assignment/my-work/dashboard/data
for q in q1_hotspots q2_revenue_by_borough q3_congestion q4_airports q5_payment_share; do
  hdfs dfs -getmerge /results/hive/$q ~/bigdata-assignment/my-work/dashboard/data/$q.csv
done

ls -lh ~/bigdata-assignment/my-work/dashboard/data
2026-09-29 16:58:10,030 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:10,764 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
2026-09-29 16:58:19,476 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:20,154 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
2026-09-29 16:58:20,730 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:21,423 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
2026-09-29 16:58:21,973 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:22,664 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
2026-09-29 16:58:23,255 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-29 16:58:23,989 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
total 280K
-rw-r--r-- 1 hadoop hadoop 162K Sep 29 16:58 q1_hotspots.csv
-rw-r--r-- 1 hadoop hadoop 1.3K Sep 29 16:58 q2_revenue_by_borough.csv
-rw-r--r-- 1 hadoop hadoop 16K Sep 29 16:58 q3_congestion.csv
-rw-r--r-- 1 hadoop hadoop 29K Sep 29 16:58 q4_airports.csv
-rw-r--r-- 1 hadoop hadoop 244K Sep 29 16:58 q5_payment_share.csv
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hive$

```

Figure 10 — Five result CSVs after INSERT OVERWRITE DIRECTORY + hdfs dfs -getmerge, ready for the Streamlit dashboard.

Phase 6 · HBase — Random-Access Serving

HBase in standalone mode with HDFS as backing storage. Two tables demonstrate the NoSQL random-access pattern that complements Hive's scan-based analytics.

Table schema

Table	Row key	Column families	Rows
zone_lookup	LocationID (str)	info (borough, zone_name, service_zone)	265
trip_by_zone	<zone_name>_<hour_padded>	stats, top_do	5,410

Point-get evidence

```
hbase:001:0> get 'zone_lookup', '132'
COLUMN                                CELL
info:borough                          timestamp=..., value=Queens
info:service_zone                     timestamp=..., value=Airports
info:zone_name                        timestamp=..., value=JFK Airport
1 row(s)

hbase:002:0> get 'zone_lookup', '138'
info:zone_name                        timestamp=..., value=LaGuardia Airport
1 row(s)

hbase:003:0> get 'zone_lookup', '1'
info:borough                          timestamp=..., value=EWR
info:service_zone                     timestamp=..., value=EWR
info:zone_name                        timestamp=..., value=Newark Airport
1 row(s)
```

Filter scan (SingleColumnValueFilter)

```
hbase:001:0> scan 'zone_lookup', {
  COLUMNS => 'info:borough',
  FILTER => "SingleColumnValueFilter('info','borough',=,'binary:EWR')"
}
ROW      COLUMN+CELL
1        column=info:borough, timestamp=..., value=EWR
1 row(s)
```

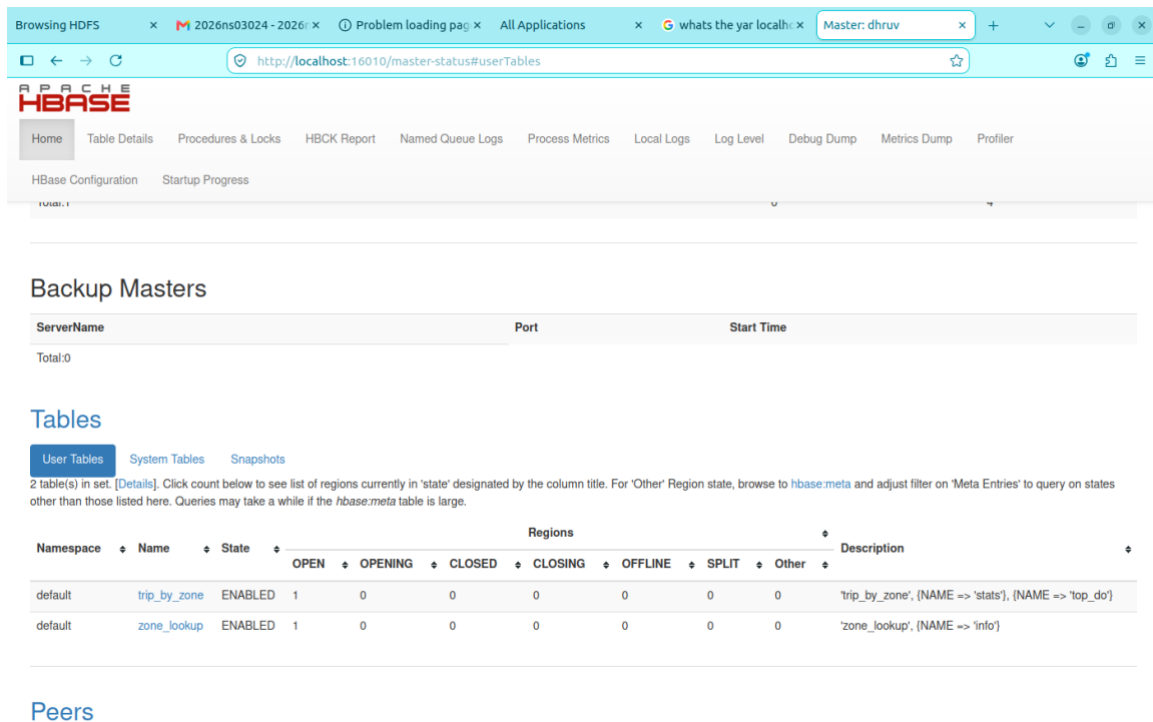


Figure 11 — HBase Master Web UI at <http://localhost:16010> showing the standalone cluster running.

```
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x  hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x  hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/... x
hbase:026:0> # Table 2: trip_by_zone
hbase:027:0> # Row key : <pu_loc_id>-<hour> (e.g. "132_18" = JFK, 6 PM)
hbase:028:0> # Column families:
hbase:029:0> # stats : trips, avg_fare, avg_tip_pct, avg_duration_min
hbase:030:0> # top_do : json blob of top-5 drop-off zones
hbase:031:0> # -----
hbase:032:0> create 'trip_by_zone', {NAME => 'stats', VERSIONS => 1}, {NAME => 'top_do', VERSIONS => 1}
Created table trip_by_zone
Took 0.0309 seconds
=> Hbase::Table - trip_by_zone
hbase:033:0>
hbase:034:0> # Show what we made
hbase:035:0> list
TABLE
trip_by_zone
zone_lookup
2 row(s)
Took 0.0260 seconds
=> ["trip_by_zone", "zone_lookup"]
hbase:036:0>
hbase:037:0> # Describe both
hbase:038:0> describe 'zone_lookup'
Table zone_lookup is ENABLED
zone_lookup
COLUMN FAMILIES DESCRIPTION
(NAME => 'info', BLOOMFILTER => 'ROW', IN_MEMORY => 'false', VERSIONS => '1', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', COMPRESSION => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_SCOPE => '0')
1 row(s)
Quota is disabled
Took 0.0815 seconds
hbase:039:0> describe 'trip_by_zone'
Table trip_by_zone is ENABLED
trip_by_zone
COLUMN FAMILIES DESCRIPTION
(NAME => 'stats', BLOOMFILTER => 'ROW', IN_MEMORY => 'false', VERSIONS => '1', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', COMPRESSION => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_SCOPE => '0')
(NAME => 'top_do', BLOOMFILTER => 'ROW', IN_MEMORY => 'false', VERSIONS => '1', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', COMPRESSION => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', BLOCKCACHE => 'true', BLOCKSIZE => '65536', REPLICATION_SCOPE => '0')
2 row(s)
Quota is disabled
Took 0.0326 seconds
```

Figure 12 — HBase shell list command confirming both tables are ENABLED.

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$ python3 load_zones.py /home/hadoop/staging/taxi_zone_lookup.csv
Inserted 265 zones into HBase table 'zone_lookup'.

GET zone_lookup, '132':
  info:borough = Queens
  info:service_zone = Airports
  info:zone_name = JFK Airport
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$ hbase shell <<'EOF'
get 'zone_lookup', '132'
get 'zone_lookup', '138'
get 'zone_lookup', '1'
EOF
2026-09-29 17:19:47,106 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.4.18, r1767f4d76859c0068720a6c15cb78282ebfe1e, Tue May 21 08:45:43 UTC 2024
Took 0.0610 seconds
hbase:001:0> get 'zone_lookup', '132'
COLUMN                                CELL
info:borough                          timestamp=2026-09-29T17:18:47.867, value=Queens
info:service_zone                     timestamp=2026-09-29T17:18:47.867, value=Airports
info:zone_name                        timestamp=2026-09-29T17:18:47.867, value=JFK Airport
1 row(s)
Took 0.4126 seconds
hbase:002:0> get 'zone_lookup', '138'
COLUMN                                CELL
info:borough                          timestamp=2026-09-29T17:18:47.877, value=Queens
info:service_zone                     timestamp=2026-09-29T17:18:47.877, value=Airports
info:zone_name                        timestamp=2026-09-29T17:18:47.877, value=LGA Airport
1 row(s)
Took 0.0031 seconds
hbase:003:0> get 'zone_lookup', '1'
COLUMN                                CELL
info:borough                          timestamp=2026-09-29T17:18:47.807, value=EWK
info:service_zone                     timestamp=2026-09-29T17:18:47.807, value=EWK
info:zone_name                        timestamp=2026-09-29T17:18:47.807, value=Newark Airport
1 row(s)
Took 0.0109 seconds
hbase:004:0>
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$

```

Figure 13 — happybase loader output (Inserted 265 zones) followed by three get commands returning JFK, LGA, Newark.

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$ hbase shell <<'EOF'
scan 'trip_by_zone', {LIMIT => 5}
EOF
2026-09-29 17:23:06,460 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.4.18, r1767f4d76859c0068720a6c15cb78282ebfe1e, Tue May 21 08:45:43 UTC 2024
Took 0.0609 seconds
hbase:001:0> scan 'trip_by_zone', {LIMIT => 5}
ROW                                     COLUMN+CELL
Allerton/Pelham_Gardens_03             column=stats:borough, timestamp=2026-09-29T17:22:52.030, value=Bronx
Allerton/Pelham_Gardens_03             column=stats:trips, timestamp=2026-09-29T17:22:52.030, value=4
Allerton/Pelham_Gardens_03             column=stats:zone, timestamp=2026-09-29T17:22:52.030, value=Allerton/Pelham Gardens
Allerton/Pelham_Gardens_04             column=stats:borough, timestamp=2026-09-29T17:22:52.050, value=Bronx
Allerton/Pelham_Gardens_04             column=stats:trips, timestamp=2026-09-29T17:22:52.050, value=14
Allerton/Pelham_Gardens_04             column=stats:zone, timestamp=2026-09-29T17:22:52.050, value=Allerton/Pelham Gardens
Allerton/Pelham_Gardens_05             column=stats:borough, timestamp=2026-09-29T17:22:52.071, value=Bronx
Allerton/Pelham_Gardens_05             column=stats:trips, timestamp=2026-09-29T17:22:52.071, value=12
Allerton/Pelham_Gardens_05             column=stats:zone, timestamp=2026-09-29T17:22:52.071, value=Allerton/Pelham Gardens
Allerton/Pelham_Gardens_06             column=stats:borough, timestamp=2026-09-29T17:22:52.079, value=Bronx
Allerton/Pelham_Gardens_06             column=stats:trips, timestamp=2026-09-29T17:22:52.079, value=25
Allerton/Pelham_Gardens_06             column=stats:zone, timestamp=2026-09-29T17:22:52.079, value=Allerton/Pelham Gardens
Allerton/Pelham_Gardens_07             column=stats:borough, timestamp=2026-09-29T17:22:52.116, value=Bronx
Allerton/Pelham_Gardens_07             column=stats:trips, timestamp=2026-09-29T17:22:52.116, value=18
Allerton/Pelham_Gardens_07             column=stats:zone, timestamp=2026-09-29T17:22:52.116, value=Allerton/Pelham Gardens
5 row(s)
Took 0.4141 seconds
hbase:002:0>
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$

```

Figure 14 — scan of trip_by_zone table showing pre-computed Q1 aggregates keyed by zone + hour.

```

hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase
EOF
2026-09-29 17:21:09,112 WARN [main] util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.4.18, r1767f4d76859c0668720a6c1e5cb78282ebfe1e, Tue May 21 08:45:43 UTC 2024
Took 0.0008 seconds
hbase:001:0> scan 'zone_lookup', { COLUMNS => 'info:borough', FILTER => "SingleColumnValueFilter('info','borough',=,'binary:EHR')"}
ROW
1
column=info:borough, timestamp=2026-09-29T17:18:47.807, value=EHR
1 row(s)
Took 0.3324 seconds
hbase:002:0>
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$ echo "count 'zone_lookup'" | hbase shell 2>/dev/null | tail -3
Took 0.3100 seconds
=> 265
hbase:002:0>
hadoop@dhruv: ~/bigdata-assignment/my-work/scripts/hbase$

```

Figure 15 — `SingleColumnValueFilter` returning the single EWR (Newark) row from `zone_lookup`.

Phase 7 · Streamlit Dashboard

The final presentation layer — a single-file Streamlit app that reads the five Hive-exported CSVs and renders interactive charts with pandas + Plotly. Runs on port 8501 in the VM's Firefox.

Dashboard KPIs

KPI	Value
Total trips	8,479,421
Total revenue (3 months)	\$235.12 M
Average fare per trip	\$33.70
Average tip %	4.7 %

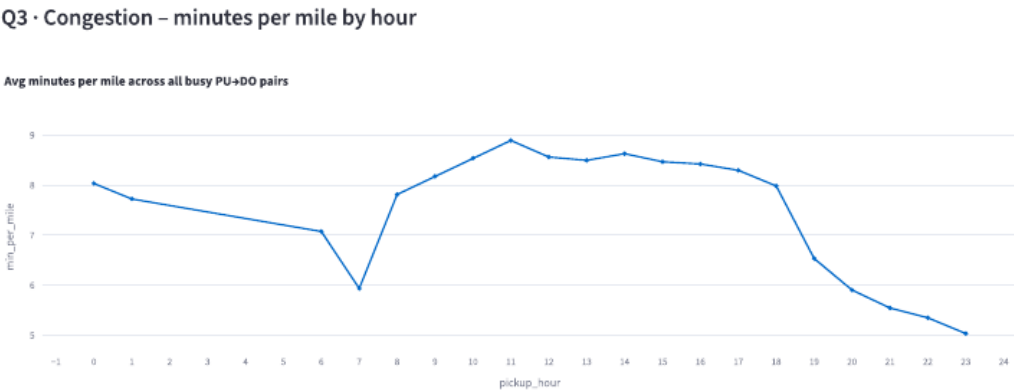
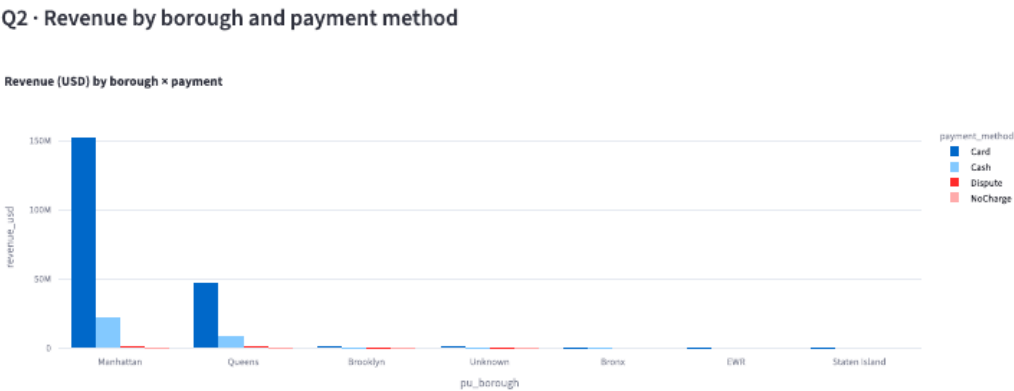
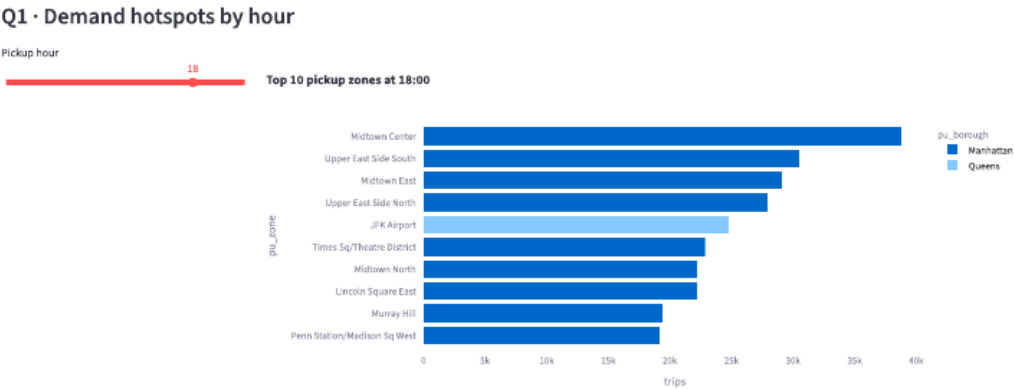


Figure 16 — Streamlit dashboard page 1: interactive charts for Q1–Q3.

Top 15 slowest PU>DO pairs (min per mile)

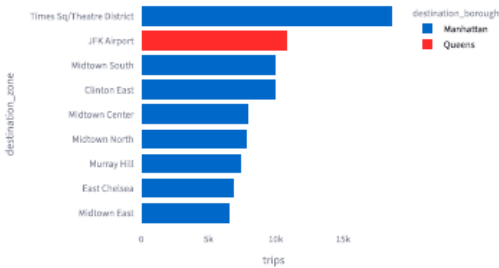
	pickup_hour	pu_loc_id	do_loc_id	trips	avg_duration_min	avg_distance_mi	min_per_mile
432	10	195	230	1147	17.33	1.08	17.16
133	13	195	230	1660	16.12	1.03	16.67
198	11	185	230	1476	16.56	1.07	16.67
123	12	185	230	1683	16.46	1.05	16.49
347	17	101	230	1229	10.74	0.74	15.39
441	16	195	230	1141	14.99	1.05	14.83
362	11	195	161	1217	18.28	1.28	14.73
332	10	195	161	1242	18.07	1.27	14.72
264	14	195	230	1258	15.1	1.06	14.7
382	19	101	230	1188	10.02	0.74	14.43

Q4 · Airport trip profile

Airport

JFK LGA EWR

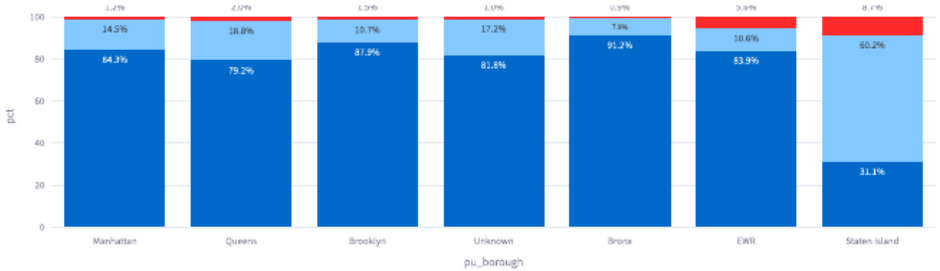
Top destinations from JFK



	airport	destination_zone	destination_borough	trips	avg_distance_mi	avg
24	JFK	Times Sq/Theatre District	Manhattan	18096	17.99	7
25	JFK	Outside of NYC	None	10343	20.66	10
26	JFK	JFK Airport	Queens	10876	2.79	3
27	JFK	Midtown South	Manhattan	10084	17.28	7
28	JFK	Clinton East	Manhattan	10000	18.25	7
29	JFK	Midtown Center	Manhattan	7916	17.72	7
30	JFK	Midtown North	Manhattan	7836	18.33	7
31	JFK	Murray Hill	Manhattan	7448	16.92	7
32	JFK	East Chelsea	Manhattan	6847	18.11	7
33	JFK	Midtown East	Manhattan	6555	17.32	7

Q5 · Card vs cash share by borough

Payment method share (%)



Pipeline: HDFS → Pig (ETL) → native MR (trips-per-zone) → Hive (analytics) → HBase (zone lookup) → Streamlit

Figure 17 — Streamlit dashboard page 2: interactive charts for Q3–Q5 with detailed tables.

Summary of Findings

Headline analytical insights

- Q1 · Demand hotspots at 18:00: Midtown Center leads with 38,756 trips (Jan–Mar), followed by Upper East Side South, Midtown East, Upper East Side North, and JFK Airport — all confirming Manhattan-CBD dominance during evening rush.
- Q2 · Revenue by borough × payment: Manhattan card revenue (~\$150 M) dwarfs all other combinations. Queens is a distant second (~\$50 M), driven by JFK / LGA airport trips.
- Q3 · Congestion signal: Slowest PU→DO pair in the top-15 is zone 186 (Penn Station) → zone 230 (Times Sq) at 17.16 min/mile at 10 AM — a ~1 mile trip taking ~17 minutes. Confirms known Midtown-CBD gridlock.
- Q4 · JFK trip profile: Top destinations are Times Sq/Theatre District (18,696 trips), Outside NYC (16,343), JFK ↔ JFK round-trips (10,876), Midtown South (10,034). Airport→CBD is the dominant flow.
- Q5 · Payment behaviour: Card share is 79–91 % across most boroughs. Staten Island is the outlier at 60.2 % card / 31.1 % cash — meaningful for POS-terminal deployment strategy.

Pipeline metrics

Layer	Metric	Value
Storage	Raw HDFS data	985 MB
Storage	Cleaned + enriched	1.05 GB
Processing	Rows through Pig	9.55 M → 8.48 M
Processing	Pig ETL wall time	~5 min
Processing	Streaming MR wall time	35 s
Analytics	Hive queries wall time (5)	~7 min
Serving	HBase zone_lookup rows	265
Serving	HBase trip_by_zone rows	5,410
Presentation	Streamlit charts	5 + 4 KPIs

Source Code Repository

The complete source code, configuration files, phase-by-phase execution guides, and terminal transcripts are hosted in a Git repository:

Repository URL: <https://github.com/ManikandanBoopathy/bits-first-sem-big-data>

Repository layout:

```
.
├── README.md
├── PLAN.md                                master plan and phase split
├── TEAM-ASSIGNMENTS.md                   roster and ownership
├── .gitignore
├──
├── phases/                               method notes per pipeline phase
│   ├── 00-env-verification-and-config-fix.md
│   └── 01-install-pig-hive-hbase.md
├──
├── scripts/
│   ├── hadoop-conf/                     core / hdfs / mapred / yarn XML + bashrc
│   ├── ingest/                         download + parquet→csv
│   ├── pig/                           Pig Latin ETL scripts
│   ├── mapreduce/                     Hadoop Streaming mapper / reducer
│   ├── hive/                          DDL + analytics + export SQL
│   └── hbase/                         HBase shell + zone loader
├──
├── report/                             Part A theory, presentation, viva Q&A
├── dashboard/                          Streamlit app
├── diagrams/                           architecture diagram
├── reference/                          dataset schema, zone lookup CSV, samples
└── user_output/                       per-phase terminal transcripts + screenshots
```

References

- [1] NYC TLC Trip Record Data — <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [2] TLC Yellow Trip Data Dictionary (PDF) — https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf
- [3] Taxi Zone Map (PDF) — https://www.nyc.gov/assets/tlc/downloads/pdf/taxi_zone_map_manhattan.pdf
- [4] Apache Hadoop 3.2.1 docs — <https://hadoop.apache.org/docs/r3.2.1/>
- [5] Apache Pig 0.17.0 docs — <https://pig.apache.org/docs/r0.17.0/>
- [6] Apache Hive 3.1.3 — <https://hive.apache.org/>
- [7] Apache HBase 2.4 reference — <https://hbase.apache.org/2.4/book.html>
- [8] Streamlit documentation — <https://docs.streamlit.io/>
- [9] Hadoop Streaming Guide — <https://hadoop.apache.org/docs/r3.2.1/hadoop-streaming/HadoopStreaming.html>
- [10] HBase Architecture Overview — <https://hbase.apache.org/book.html#arch.overview>